

Feature Engineering

Turning data into useful features

prepare • train • predict • evaluate • improve



A concise practical introduction

1. Feature Engineering

What

Process of selecting, creating, or transforming features (input variables) in your dataset

Why?

Plays a pivotal role in model accuracy, interpretability, and generalization

How?

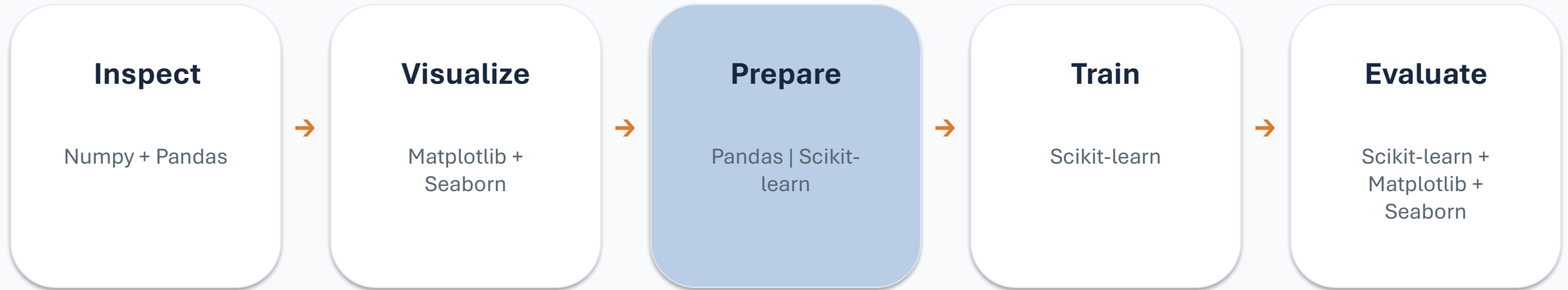
Creative and iterative process that requires domain knowledge and understanding of the problem

<https://scikit-learn.org/stable/api/sklearn.preprocessing.html>

Raw data is not necessarily model-ready

2. Where does feature engineering fit?

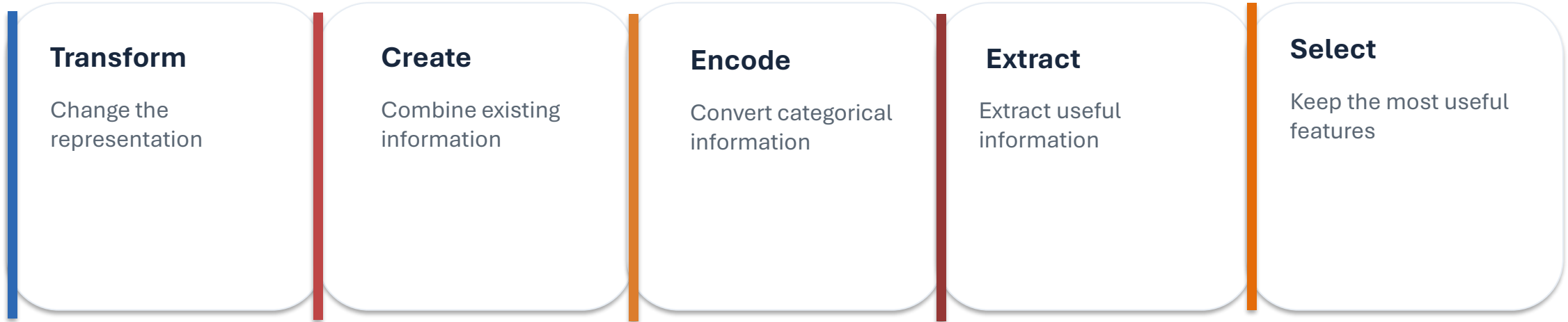
Use feature engineering before model fit and evaluation.



From data to machine learning

3. Types of Feature Engineering

A model is useful only if its predictions are good.



4. Encoding

Almost every estimator follows the same basic idea:

```
from sklearn.preprocessing import
OneHotEncoder

enc = OneHotEncoder()
enc.fit_transform(X_train)

enc.transform(X_test)
```

OneHotEncoder()

Categories -> Binary Vector

OrdinalEncoder()

Categories -> Ascending values

LabelEncoder()

Target categories -> 0 to number of classes - 1

5. Scaling

Some algorithms are sensitive to feature scale.

```
from sklearn.preprocessing import  
StandardScaler  
  
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)
```

StandardScaler()

Standardize features by removing the mean and scaling to unit variance.

MaxAbsScaler()

Scale each feature by its maximum absolute value.

MinMaxScaler()

Transform features by scaling each feature to a given range.

Normalizer()

Normalize samples individually to unit norm.

6. Impute

Many models cannot directly handle missing values.

```
from sklearn.impute import SimpleImputer  
  
imputer = SimpleImputer(strategy="median")  
  
X_imputed = imputer.fit_transform(X)
```

SimpleImputer()

Univariate imputer for completing missing values with simple strategies.

KNNImputer()

Imputation for completing missing values using k-Nearest Neighbors.

IterativeImputer()

Multivariate imputer that estimates each feature from all the others.

7. Feature Creation

Sometimes the combination of two features is more informative than either feature alone.

```
from sklearn.preprocessing import  
PolynomialFeatures  
  
poly = PolynomialFeatures(degree=2)  
  
X_poly = poly.fit_transform(X)
```

Interaction Features

Created by performing mathematical operations (e.g., multiplication, division) on feature pairs.

Polynomial Features

Might improve model performance when the true relationship between features and the target variable is nonlinear.

Warning

More features $\neq$ better model.

Polynomial expansion can produce a very large number of features.

8. Feature Selection

Sometimes we have too many features.

```
from sklearn.feature_selection import  
SelectKBest, f_classif  
  
selector = SelectKBest(f_classif, k=4)  
selector.fit(X_train, y_train)  
scores = -np.log10(selector.pvalues_)  
scores /= scores.max()
```

VarianceThreshold()

Feature selector that removes all low-variance features.

SelectKBest()

Select features according to the k highest scores.

9. Feature engineering and data leakage

This is one of the **most important concepts**.

Data Leakage

Information that would not legitimately be available at prediction time influences the model.

Suppose we want to predict whether a student will pass.

Attendance
Assignments
Midterm grade
Final grade

Is "final grade" a good feature for predicting whether the student passed?